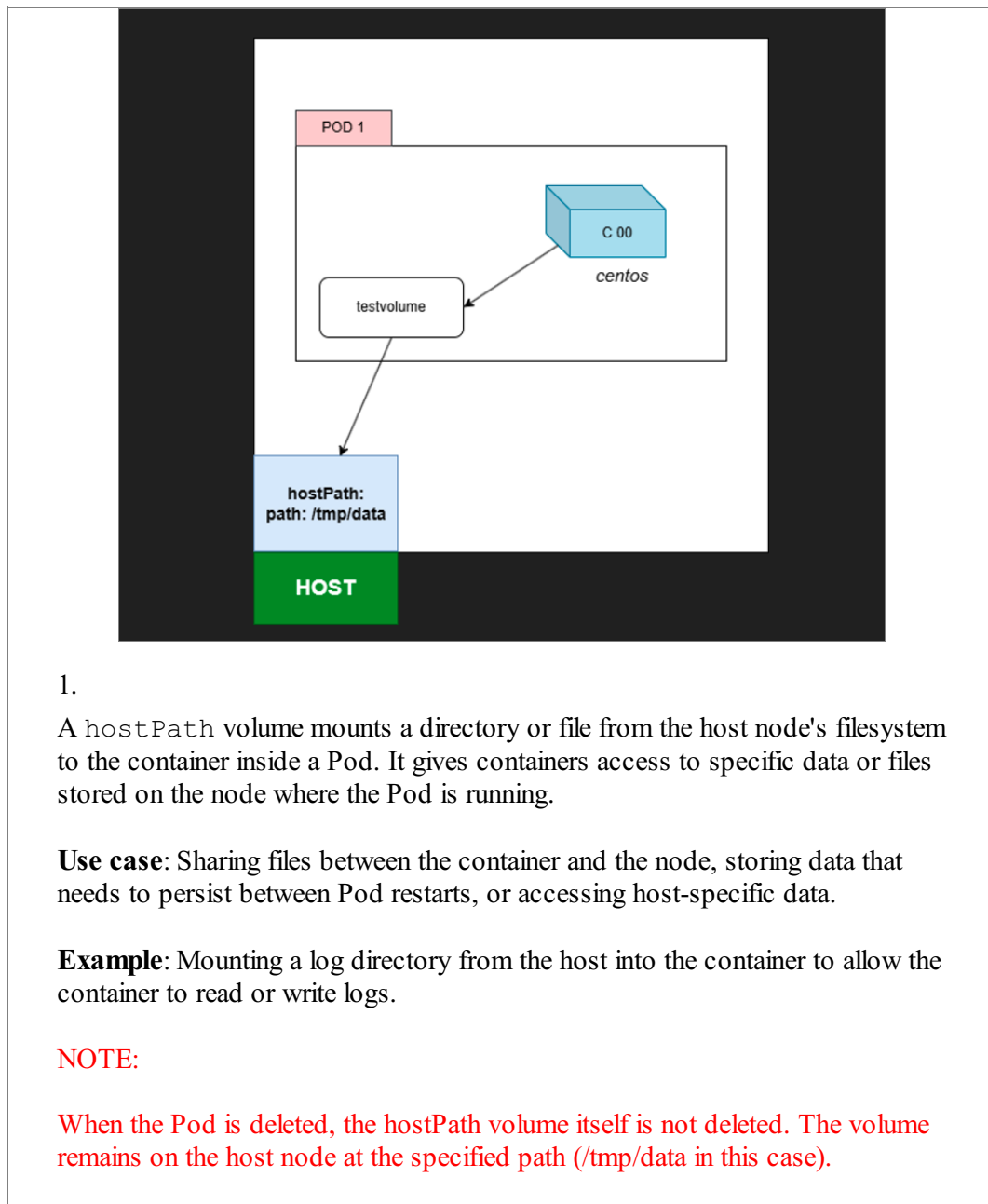

Volumes in Kubernetes(HostPath)

Rohit Surwade

HostPath



Difference between HostPath and EmptyDir

Feature	hostPath	emptyDir
Data Persistence	Persists after Pod deletion	Lost when Pod is deleted
Storage Location	On the host node's filesystem	On the node's local storage
Usage	Mounts specific files or directories from the host	Temporary storage for Pod data
Security	Potential security risks (exposes host filesystem)	More secure since it's managed by Kubernetes
Data Sharing	Can share files between the container and the host	Can be used for sharing data between containers in the same Pod
Flexibility	Can mount arbitrary host directories/files	Always starts empty and temporary
Best Use Case	Accessing host files, logs, or devices	Temporary data, caching, or scratch space

2.

A Manifest to create a pod along with "HostPath" volume

```

root@ip-172-31-47-35: /home/ubuntu
apiVersion: v1
kind: Pod
metadata:
  name: myvolhostpath
spec:
  containers:
    - name: testc
      image: centos
      command: ["/bin/bash", "-c", "sleep 15000"]
      volumeMounts:
        - name: testvolume
          mountPath: /tmp/hostpath
  volumes:
    - name: testvolume
      hostPath:
        path: /tmp/data

```

3.

(A Pod named myvolhostpath is created with one container (testc). The container uses the centos image and runs a sleep command to keep it alive. The container has a volume, testvolume, which is of type hostPath. The hostPath volume mounts the directory /tmp/data from the host machine into the container at /tmp/hostpath. hostPath is typically used when you need to share data between a container and the host machine. In this case, the container testc will have access to the /tmp/data directory on the host, and any changes made to /tmp/hostpath inside the container will directly affect the /tmp/data directory on the host. However, it's important to note that using hostPath volumes ties your Pod to a specific node, which can affect the portability of your workloads. In production environments, other volume types like PersistentVolume or NFS are often preferred because they abstract away the specifics of the host node.

hostPath volumes behave differently than other volume types like Persistent Volumes (PVs) because they are tied directly to the host node where the Pod is running. When the Pod is deleted, the hostPath volume itself is not deleted. The volume remains on the host node at the specified path (/tmp/data in this case) even after the Pod is gone. The directory /tmp/data on the host machine will still exist, and any files stored there will remain intact. However, since the Pod is gone, the container will no longer have access to this volume unless a new Pod mounts the same hostPath volume and points to the same path on the host node.)

```
apiVersion: v1
kind: Pod
metadata:
  name: myvolhostpath
spec:
  containers:
  - name: testc
    image: centos
    command: ["/bin/bash", "-c", "sleep 15000"]
    volumeMounts:
    - name: testvolume
      mountPath: /tmp/hostpath
  volumes:
  - name: testvolume
    hostPath:
      path: /tmp/data
```

Our POD is up and running

```

root@ip-172-31-47-35:/home/ubuntu# cat hostpath.yml
root@ip-172-31-47-35:/home/ubuntu# kubectl apply -f hostpath.yml
pod/myvolhostpath created
root@ip-172-31-47-35:/home/ubuntu# kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
myvolhostpath       1/1     Running   0           5m55s
root@ip-172-31-47-35:/home/ubuntu#

```

4.

Lets enter the container(testc) and check if the volume is there

```

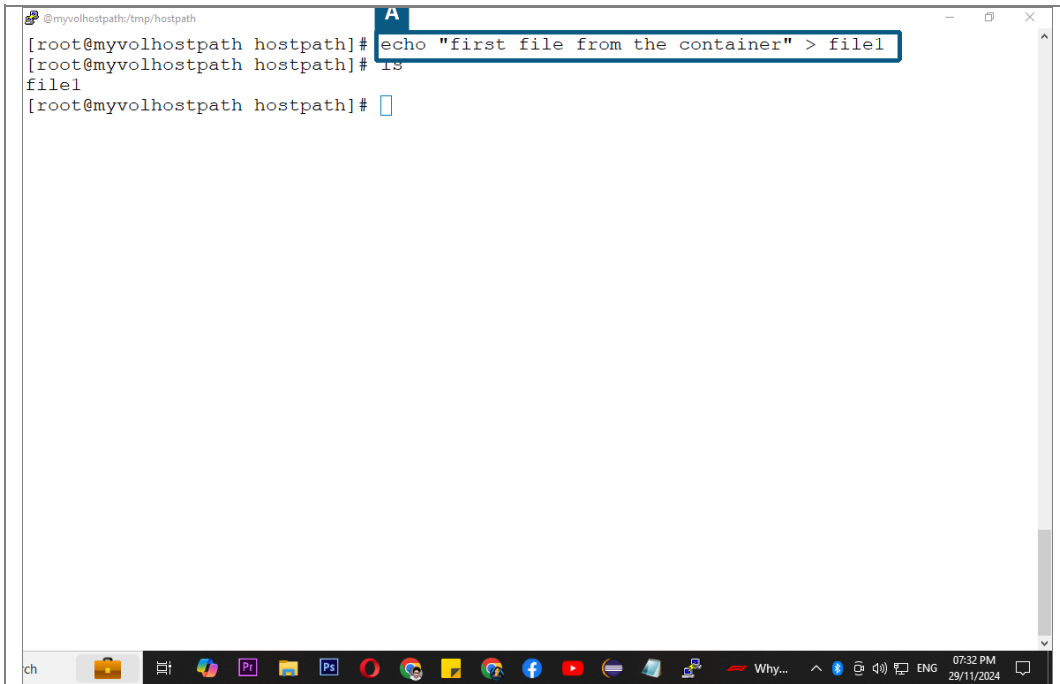
root@ip-172-31-47-35:/home/ubuntu# vi hostpath.yml
root@ip-172-31-47-35:/home/ubuntu# kubectl apply -f hostpath.yml
pod/myvolhostpath created
root@ip-172-31-47-35:/home/ubuntu# kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
myvolemptydir       2/2     Running   0           62m
myvolhostpath       1/1     Running   0           5m55s
root@ip-172-31-47-35:/home/ubuntu# kubectl exec myvolhostpath -it -c testc -- /bin/bash
[root@myvolhostpath /]# ls
bin  etc  lib  lost+found  mnt  proc  run  srv  tmp  var
dev  home  lib64  media      opt  root  sbin  sys  usr
[root@myvolhostpath /]# cd tmp
[root@myvolhostpath tmp]# ls
hostpath  ks-script-4luisyla  ks-script-o23i7rc2  ks-script-x6ei4wuu
[root@myvolhostpath tmp]# cd hostpath/
[root@myvolhostpath hostpath]# ls
[root@myvolhostpath hostpath]#

```

5.

The volume "testvolume" is there in "hostpath" directory

Lets create a file in the volume from the container



A terminal window titled '@myvolhostpath/tmp/hostpath' showing the following commands and output:

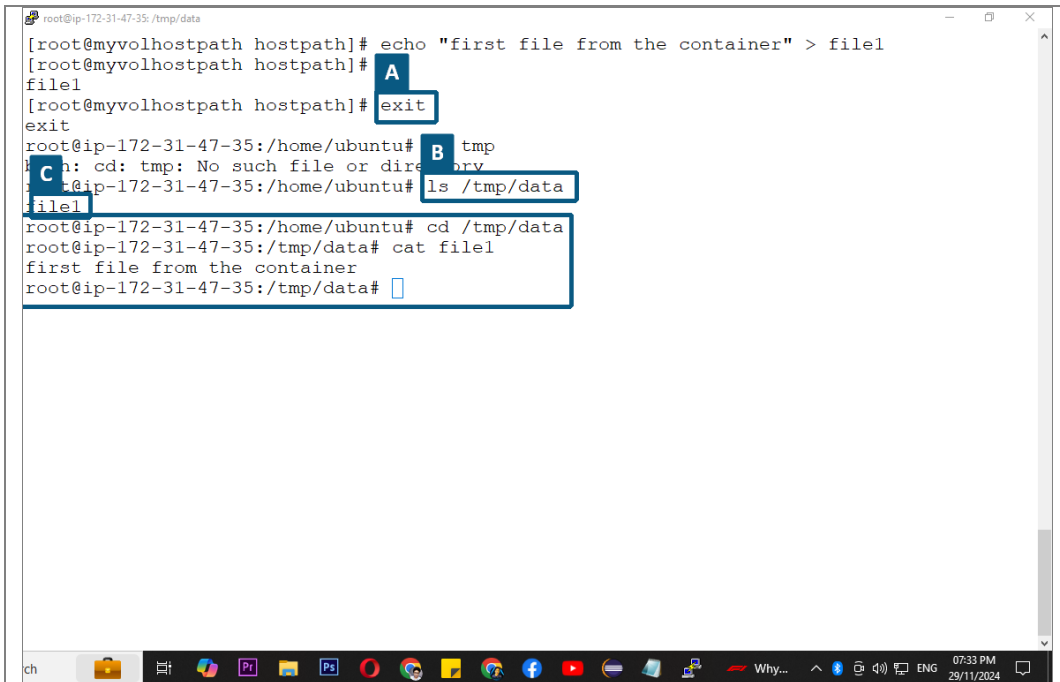
```
[root@myvolhostpath hostpath]# echo "first file from the container" > file1
[root@myvolhostpath hostpath]# ls
file1
[root@myvolhostpath hostpath]#
```

The terminal window has a taskbar at the bottom with various application icons and a system tray showing the time as 07:32 PM on 29/11/2024.

6.

echo "content" > file-name

Lets exit the container and check if files are reflecting in host node at the specified path (/tmp/data)



A terminal window titled 'root@ip-172-31-47-35: /tmp/data' showing the following commands and output:

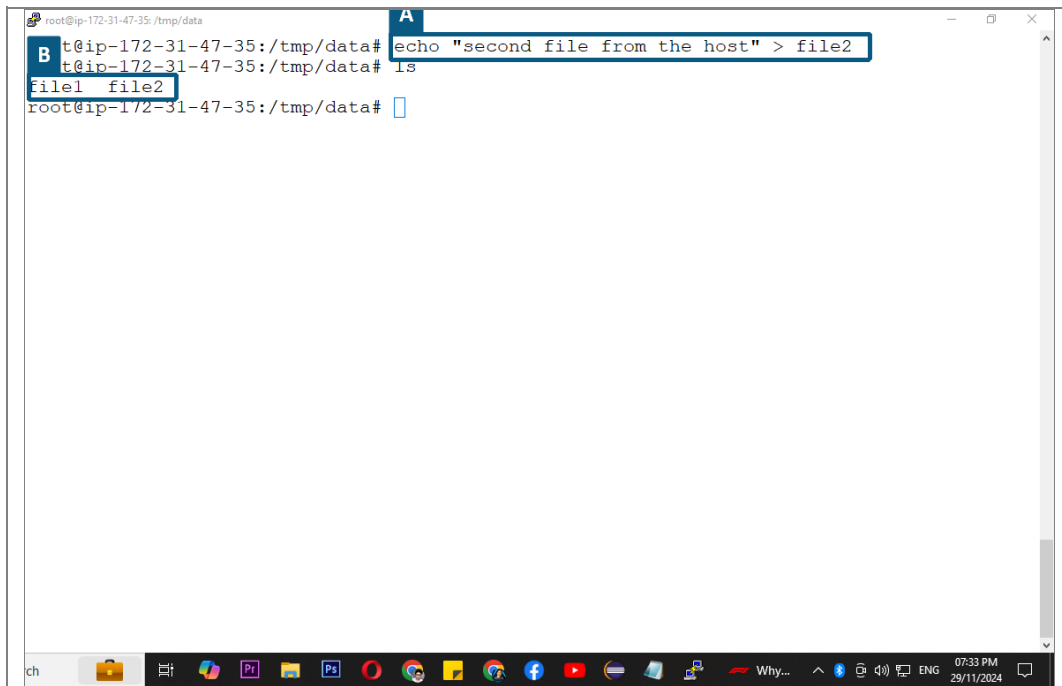
```
[root@myvolhostpath hostpath]# echo "first file from the container" > file1
[root@myvolhostpath hostpath]# ls
file1
[root@myvolhostpath hostpath]# exit
exit
root@ip-172-31-47-35:/home/ubuntu# cd /tmp
root@ip-172-31-47-35:/tmp# ls /tmp/data
file1
root@ip-172-31-47-35:/home/ubuntu# cd /tmp/data
root@ip-172-31-47-35:/tmp/data# cat file1
first file from the container
root@ip-172-31-47-35:/tmp/data#
```

The terminal window has a taskbar at the bottom with various application icons and a system tray showing the time as 07:33 PM on 29/11/2024.

7.

Data is reflecting

Creating a new file in the volume from the Host



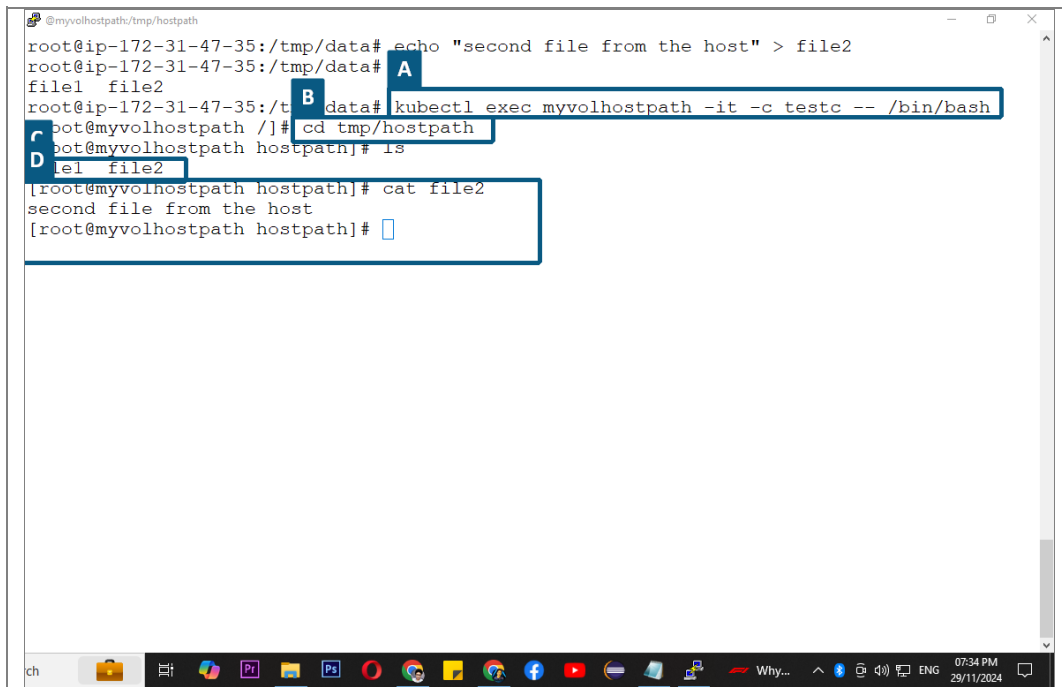
A terminal window showing a root user on a host with IP 172-31-47-35. The user is in the /tmp/data directory. They create a file named 'file2' with the content 'second file from the host' using the command 'echo "second file from the host" > file2'. They then list the directory contents, showing 'file1' and 'file2'. The terminal window has a Windows taskbar at the bottom with various application icons and a system clock showing 07:33 PM on 29/11/2024.

```
root@ip-172-31-47-35:/tmp/data# echo "second file from the host" > file2
root@ip-172-31-47-35:/tmp/data# ls
file1 file2
root@ip-172-31-47-35:/tmp/data#
```

8.

echo "content" > file-name

Lets check if the same data is reflecting in the container



A terminal window showing a root user on a host with IP 172-31-47-35. The user is in the /tmp/data directory. They create a file named 'file2' with the content 'second file from the host' using the command 'echo "second file from the host" > file2'. They then list the directory contents, showing 'file1' and 'file2'. They then use 'kubectl exec' to run a container named 'testc' with the command 'cd tmp/hostpath'. Inside the container, they list the directory contents, showing 'file1' and 'file2'. They then use 'cat' to view the contents of 'file2', which shows 'second file from the host'. The terminal window has a Windows taskbar at the bottom with various application icons and a system clock showing 07:34 PM on 29/11/2024.

```
root@ip-172-31-47-35:/tmp/data# echo "second file from the host" > file2
root@ip-172-31-47-35:/tmp/data# ls
file1 file2
root@ip-172-31-47-35:/tmp/data# kubectl exec myvolhostpath -it -c testc -- /bin/bash
root@myvolhostpath /]# cd tmp/hostpath
root@myvolhostpath hostpath]# ls
file1 file2
root@myvolhostpath hostpath]# cat file2
second file from the host
root@myvolhostpath hostpath]#
```

9.

Data is reflecting